WEB SERVICE WITH JWT AUTHENTICATION AND AUTHORIZATION

INTRODUCTION

A Web service is a software system that supports interoperable machine - to - machine interaction over a network. It has an interface described in a machine processable format(specifically, web Service Definition Language, or WSDL). Web services fulfill a specific task or a set of tasks.A web service is described using a standard , formal XML notion , called its service description, that provides all of the details necessary to interact with the service, including message formats(that detail the operations),transport protocols and location.

JSON Web Token is an open standard(RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed.In its compact form, JSON Web Tokens consists of three parts separated by dots(.), which are:

- Header
- Payload
- Signature

OBJECTIVE

To design a web service with JWT authentication and authorization and test it using Postman.

REQUIREMENTS

1. PyCharm IDE
   Modules used:

i. Flask-
   It is a web framework for Python,allowing us to build web applications. It provide the core functionality for building web applications. It is used to define routes, handle HTTP requests , and return JSON responses.

ii. jwt-
   JWT is a compact,URL-safe means of representing claims to be transferred between two parties.It used for authentication and authorization in web applications.It is used to encode and decode JWTs.

iii. Json-
   In Python it is used for encoding and decoding JSON data.It is used to load user data from a JSON file and save it back after a new user is registered.

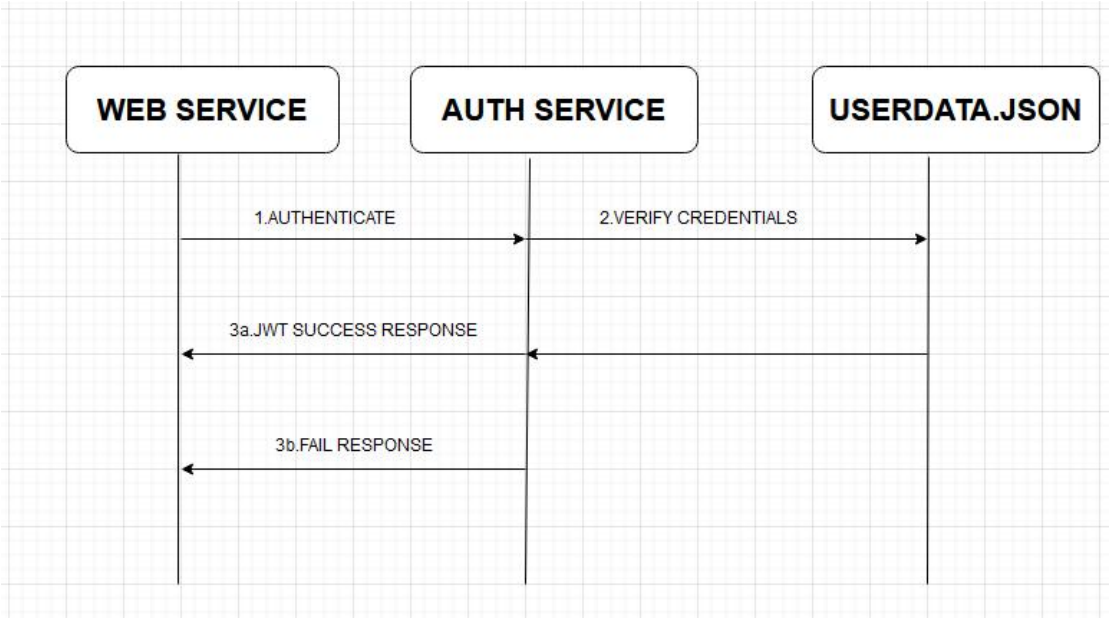
iv. Blueprint class-
   The Blueprint class is used to create modular components in a Flask application.It helps in structuring by grouping related routes and functionality together.

v. 'request' Object (from Flask)-
   The 'request' object is provided by Flask and allows us to access data submitted in an HTTP request.

vi. ‘jsonify’ Function(from Flask)-
The ‘jsonify’ function is used to convert Python dictionaries into JSON-formatted responses.It simplifies the process of returning JSON responses from Flask routes.

2. Postman
3. Terminal/Command Prompt

ARCHITECUTRE



IMPLEMENTATION

Below is the implementation of authentication and authorization that generates JSON Web Tokens.

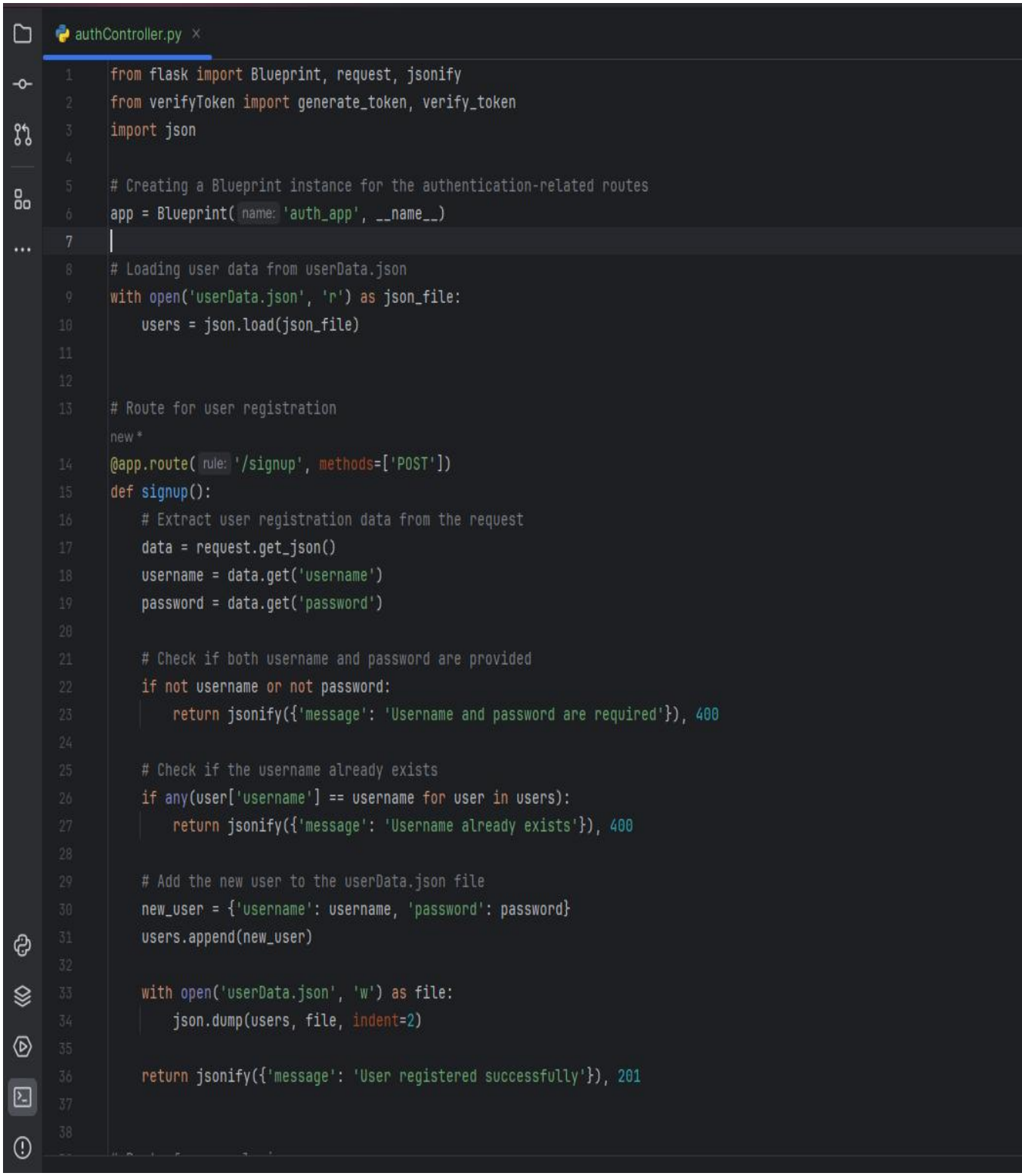
- First we create authController.py that represents a simple authentication system using Flask, which follows a modular architecture.
  Break down of the architecture:

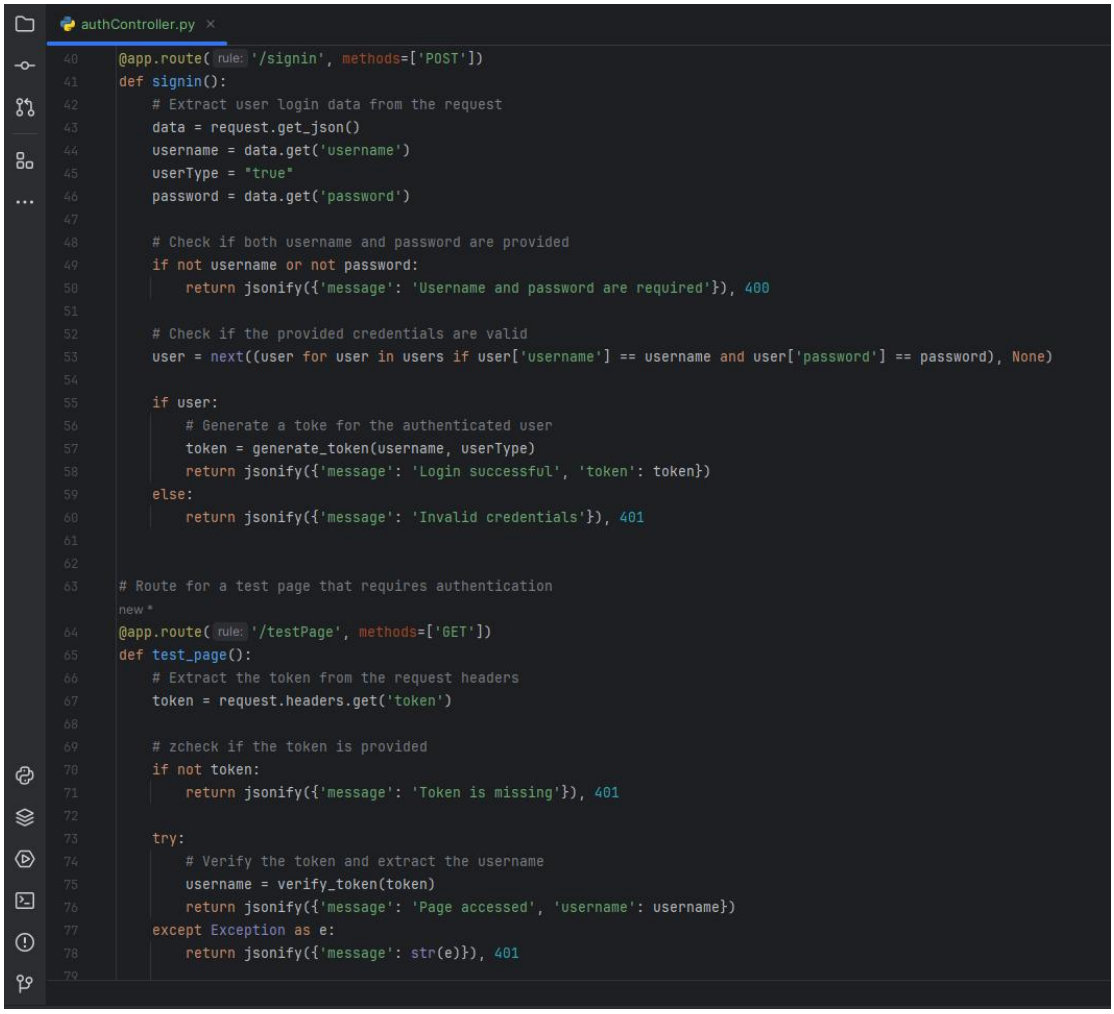
1. Flask Application and Blueprint-
   The code defines a Flask Blueprint named ‘auth_app’.Blueprints are used to organize a set of related routes and functionality in a modular way.
   The ‘app’ Blueprint encapsulates authentication-related routes such as user registration (‘/signup’),user login(‘/signin’), and a test page that requires authentication (‘/testPage’)

2. User Data Storage-
   User data is stored in a JSON file named ‘userData.json’.This file is read and loaded into memory when the Flask application starts.
3. User Registration(‘/signup’ Route)-
   Handles HTTP POST requests for user registration.
   Validates that both a username and password are provided in the JSON payload.
   Checks if the provided username already exists inn the loaded user data.

If Validation passes,a new user is added to the in-memory user data, and the data is saved back to ‘userData.json’.
Returns a JSON response indicating successful user registration or an error response if validation fails.

4. User Login(‘/signin’ Route)-
   Handles HTTP POST requests for user login.
   Validates that both a username and password are provided in the JSON payload
   Checks if the provided credentials match any user in the loaded user data.
   If valid, generates a token for the authenticated user using the ‘generated_token’ function
   Returns a JSON response indicating successful login with the generated token or an error response if validation fails.
5. TestPage(‘/testPage’ Route)-
   Handles HTTP GET requests for a test page that requires authentication.
   Extracts the token from the request headers.
   Checks if the token is provided.If not, returns an error response indicating a missing token.
   Attempts to verify the token using the ‘verify_token’ function.
   If successful, returns a JSON response indicating successful access to the page along with the username.If verification fails,returns an error response.
6. Token Generation and Verification-
   The code references two functions, ‘generate_token’ and ‘verify_token’, for token generation and verification, respectively.Token generation typically involves creating a JWT, and verification involves checking the validity of a JWT.


authController.py
from flask import Blueprint, request, jsonify
from verifyToken import generate_token, verify_token
import json
# Creating a Blueprint instance for the authentication-related routes
app = Blueprint(name: 'auth_app', __name__)
# Loading user data from userData.json
with open('userData.json', 'r') as json_file:
users = json.load(json_file)
# Route for user registration
new *
@app.route(rule: '/signup', methods=['POST'])
def signup():
# Extract user registration data from the request
data = request.get_json()
username = data.get('username')
password = data.get('password')
# Check if both username and password are provided
if not username or not password:
return jsonify({'message': 'Username and password are required'}), 400
# Check if the username already exists
if any(user['username'] == username for user in users):
return jsonify({'message': 'Username already exists'}), 400
# Add the new user to the userData.json file
new_user = {'username': username, 'password': password}
users.append(new_user)
with open('userData.json', 'w') as file:
json.dump(users, file, indent=2)
return jsonify({'message': 'User registered successfully'}), 201

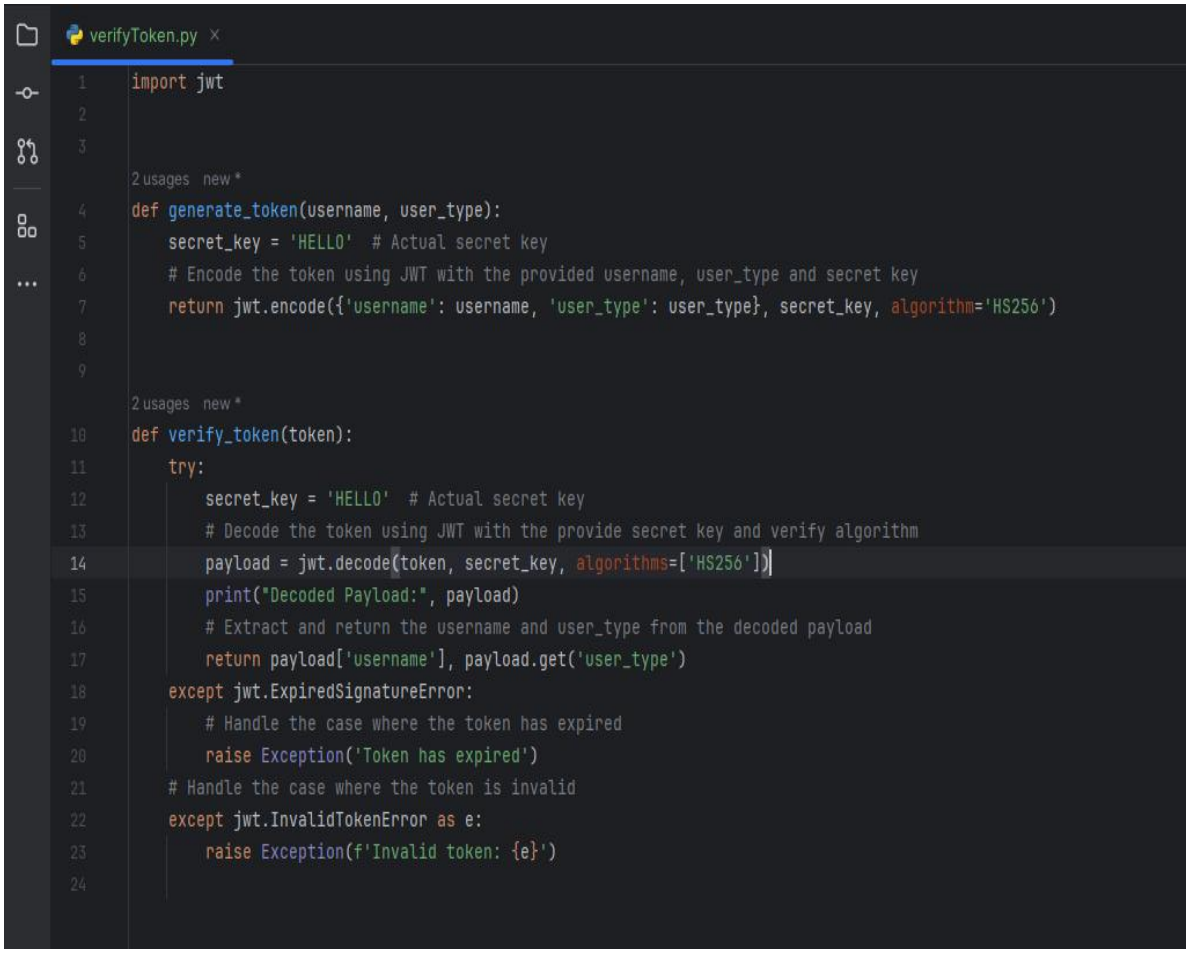


- Second, we create the verifyToken.py that defines two functions related to JWT(JSON Web Token) processing: ‘generate_token’ and ‘verify_token”.

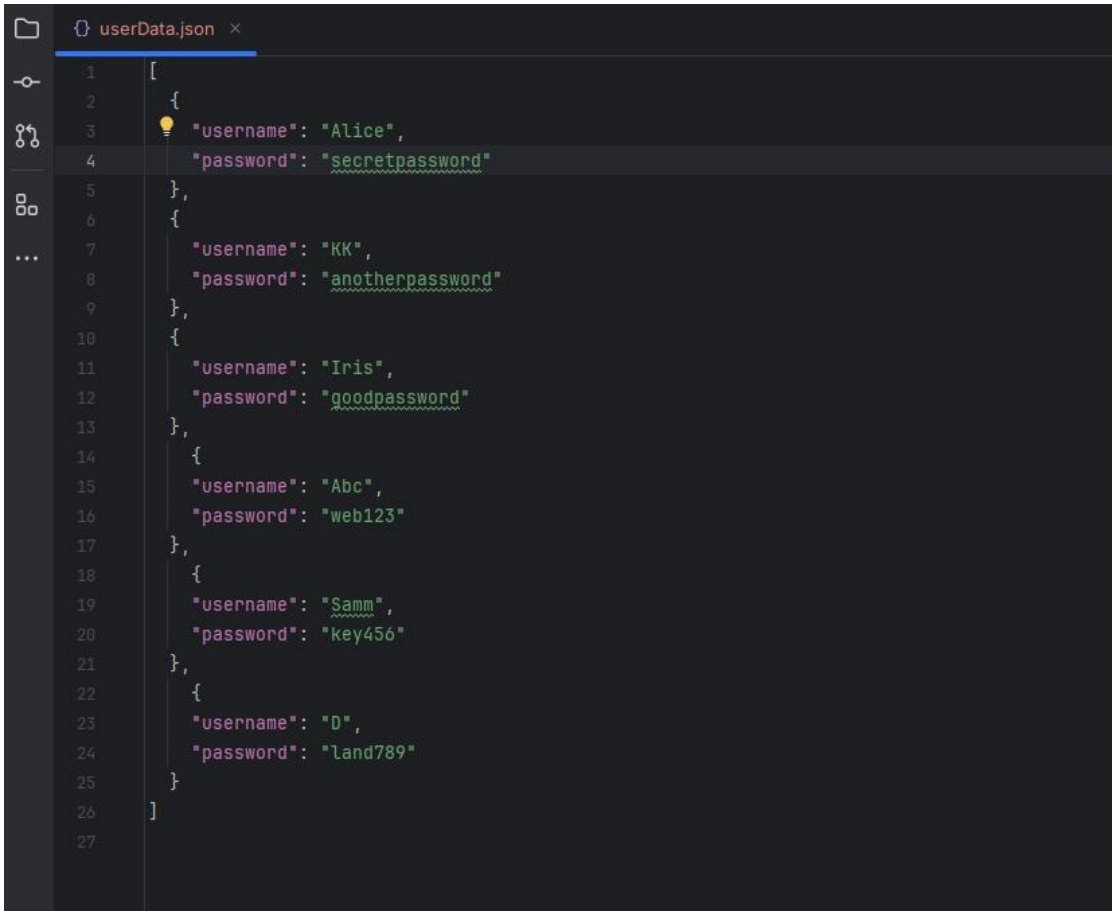
1. ‘generate_token’ Function-
   Generates a JWT token based on the provided ‘username’ and ‘user_type’.
   The function takes ‘username’ and ‘user_type’ as parameters.
   A secret key (‘HELLO’) is used to sign the token
   The ‘jwt.encode’ function from the ‘jwt’ library is used to encode the payload (containing username and user_type) into a token using the specified secret key and algorithm(‘HS256’)
   The encode token is returned.

2. ‘verify_token’ Function-|
   The function takes a ‘token’ as a parameter.
   A secret key(‘HELLO’) is used to verify the token’s signature.
   The ‘jwt.decode’ function from the ‘jwt’ library is used to decode the token, extracting the payload and checking the signature’s validity.
   The decoded payload is printed for demonstration purposes.
   The ‘jwt.ExpiredSignatureError’ exception is caught to handle the case where the token has expired.

The ‘jwt.InvalidTokenError’ exception is caught to handle other invalid token scenarios.
If the verification is successful, the ‘username’ and ‘user_type’ are extracted from the payload and returned.



- Third, we provide JSON data to represent a list of user credentials, specifically usernames and passwords
  The data structure is a list of dictionaries, where each dictionary represents a user with a username-password pair. Each user has a unique combination of username and a password.



- Last, we create the main entry point for a Flask application.

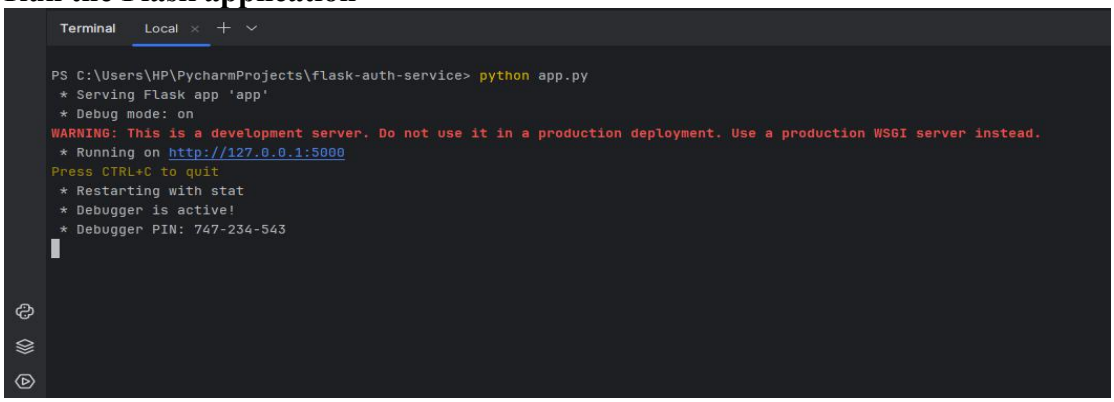
1. Flask Application Instance-
   We create a Flask application instance named ‘app’
2. Blueprint Registration-
   Registers the ‘authController’ Blueprint with the main application.
   ’auth_app’ is a Blueprint instance defined in the ‘authController’ module.\
3. Development Server Run-
   The code includes a block that runs the application when the script is executed directly.This block is wrapped with ‘if __name__ == ‘__main__’:’.

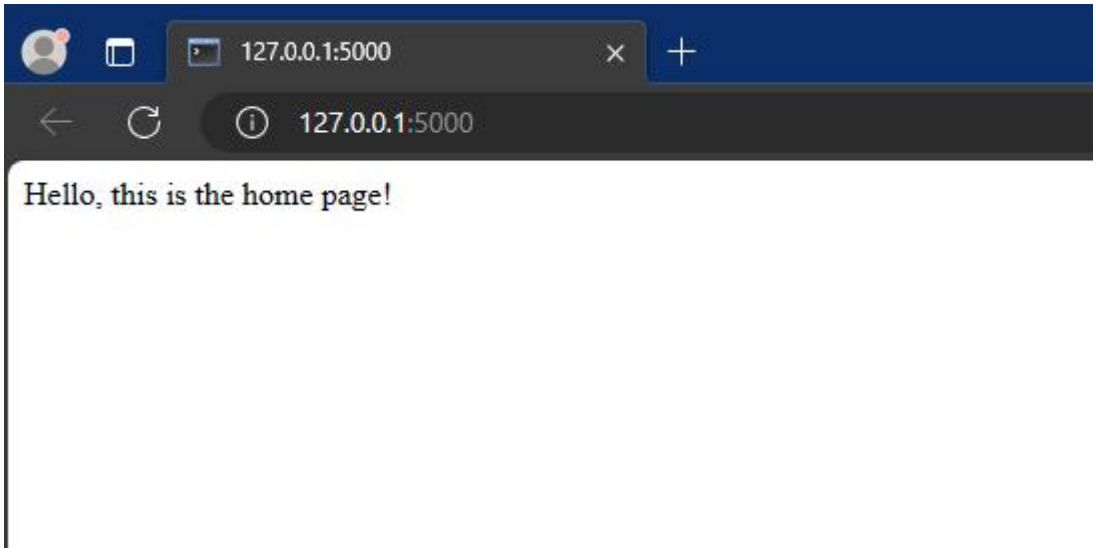
app.py

```python
from flask import Flask
from authController import app as auth_app

# A Flask application instance
app = Flask(__name__)

# Registering the authController Blueprint with the main application
app.register_blueprint(auth_app)


# Route for the home page
new *
@app.route("/")
def home():
    return "Hello, this is the home page!"


# Run
if __name__ == '__main__':
    # Enable debugging mode for development
    app.run(debug=True)
```

TESTING

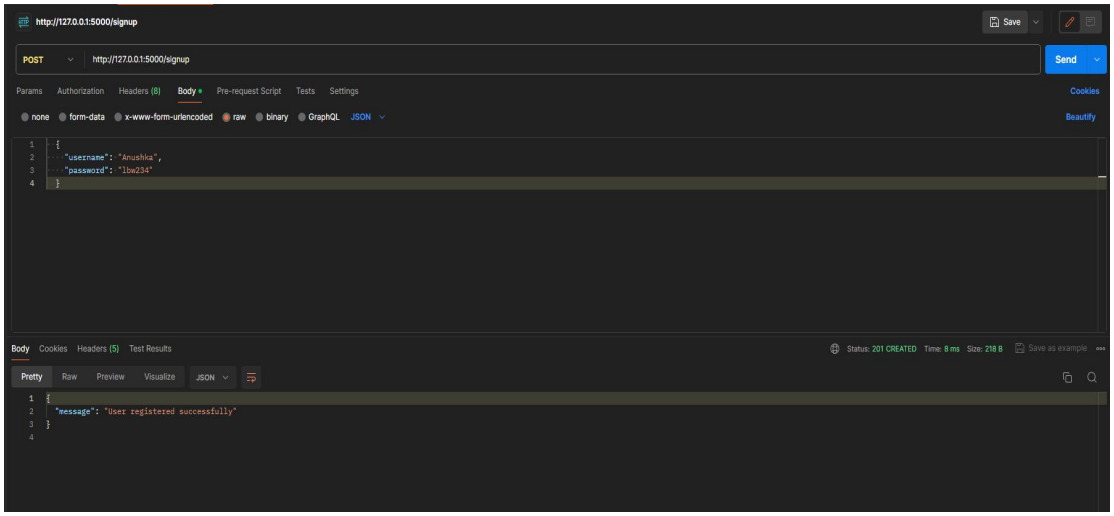
Run the Flask application-





Open Postman

1. Route for signup



2. Route for signin

3. Route for Test Page